

A FIRST-ORDER TOOL FOR GRAPH THEORY

PETER GLENN

ABSTRACT. *FlagCalc* is an open-source C++ tool for investigating graph theoretical questions on large or small databases of graphs using a custom first-order querying language that verifies conjectures in extremal graph theory, or produces statistics about criteria “in the field”: on millions of random graphs, or on individually-chosen small databases of sample graphs.

1. INTRODUCTION

Graph Theory merges with first-order queries via an open source command-line and graphical interface tool coded for multithreaded efficiency and query language expressiveness. Basic use revolves around very general options: on the one hand, a “workspace” of graphs, chosen randomly in the millions or more (i.e. typically under thirty vertices), or small workspaces of chosen sample graphs; the “workspace” then merges with several features, most typically sorting and grouping by isomorphism class, or answering any first-order query in an extremely versatile language, building on “measures” or “criteria” coded either natively in C++, or in user-supplied Python extensions, “stored procedures” (i.e. functions written in the tool’s first order language, separated out functionally for readable code and duplicate invocations).

The tool is designed to assist in proof, not via “proof search” (or any tractable portion of proof search), but via exploring and affirming or negating conjectures on a sample space, as stated, often in the millions of random graphs. The randomizer itself, of course, is extensible, with several built-in randomizers, most typically the edge-incidence specified as a decimal or count, and the tool then supplying the requested number of graphs. For isomorphism-invariant queries (i.e. most of any research interest), one can first classify by isomorphism class, then apply the quer(ies) just to one graph per isomorphism class. Or, one can reverse that and obtain all graphs that pass a given query, subsequently then binning them into isomorphisms classes.

More, queries are “staged”, so one premise whittles down the sample space by some factor (explicitly measured and recorded); a second premise further whittles down; and so on. A variant on this “staging” is the ability to output to a data file the resulting database of graphs that “passed” (or both pass and fail, with the criteria or measure indicated in the comment section preceding each graph’s data).

The tool’s success lies in its combination of an expressive and human-readable querying language, with utterly speed-minded C++ native code, that uses threads throughout, that

has some first tested GPU extensions, as well. The aim indeed is that it be useful both for a simple investigation into, say, graph planarity, Hamiltonian cycles, Chromatic number and algorithms greedy or ideal, or other most introductory features of the subject, but for serious researchers, the ability to evolve and capture any number of uses well beyond those envisioned by the author.

One example that provided motivation was Mantel’s Theorem, stating roughly that a bipartite graph provides the maximum number of edges in a triangle-free graph; this is “extremal graph theory”, and forms the basis for investigations into “flag algebras”. Flagcalc’s contribution, initially at least, is simply to posit some large number of random graphs, then to give the criteria “triangle free”, and to output how many exceed the stated number of edges (i.e., always zero). The ways to specify “triangle-free” include the direct criteria as coded in C++, C++ coded criteria that check for embeddings of a complete graph on three vertices, C++ code on the minimal cycle length found in the graph (aka the graph’s girth), or indeed just the possibility of coding a randomizer that directly outputs only triangle-free graphs.

Another use lies in “tables” of probabilities: “On a hundred million random graphs of twenty vertices, what is the likelihood if the graph is connected, that its radius be at most n ?”.

Connections to the model theory of graphs are abundant. The tool is borne out in the hope that these types of numbers and statistical likelihoods, will give meaningful data to researchers.

2. CASE STUDY: HAMILTONIAN GRAPHS

A graph is *Hamiltonian* if it contains a Hamiltonian cycle. A Hamiltonian cycle is a cycle that visits every vertex on the graph once (and a cycle, unlike a walk, visits each vertex at most once). All graphs are simple, undirected.

2.1. Expressing the existence of a Hamiltonian cycle in Flagcalc. Here are four equivalent mathematical expressions for this condition.

- (1) There is a permutation p of the graph’s vertices $V(G)$ such that every pair of p -adjacent vertices has an edge between them, as does the first and last vertex in the permutation
- (2) There is an embedding of the cyclic graph C_n on n vertices into G (a graph *embedding* is an isomorphism with a subgraph; a *subgraph* is a subset of $V(G)$ and a subset of $E(G)$ only consisting of edges between vertices in the chosen subset of $V(G)$; a subgraph is not the same thing as an *induced subgraph*, the latter preserving non-adjacency in addition to adjacency).
- (3) there is a cycle beginning and ending at the first vertex, which has length equal to the graph’s dimension $|V|$.
- (4) The circumference of the graph is equal to its dimension.

These four formalizations are chosen to illustrate the expressive simplicity of the Flagcalc querying language.

```

PTM=${PTM:-'./bin'}

# Hamiltonian cycles, in four distinct ways
$Pth/flagcalc -r 8 p=0.5 100 -a s="EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm])))" all -v i=minimal3.cfg
$Pth/flagcalc -r 8 p=0.5 100 -a s="EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$Pth/flagcalc -r 8 p=0.5 100 -a s="EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm]))) s="EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$Pth/flagcalc -r 8 p=0.5 100 -a s="EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm]))) IFF EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$Pth/flagcalc -r 8 p=0.5 100 -a s="embedsgenerousc(\"-abcdefg-h\")" all -v i=minimal3.cfg
$Pth/flagcalc -r 8 p=0.5 100 -a s="embedsgenerousc(\"-abcdefg-h\") IFF EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$Pth/flagcalc -r 8 p=0.5 100 -a s="circm == dimm" all -v i=minimal3.cfg
$Pth/flagcalc -r 8 p=0.5 100 -a s="circm == dimm IFF EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg

```

Scanning the invocations of Flagcalc in the screenshot above, we see eight invocations teasing out the four characterizations just given above. In all eight invocations the first task is to populate the “workspace” with graphs, here drawn randomly on eight vertices with an average likelihood of 0.5 edge incidence between any two vertices, and sampling one hundred such graphs. That being done, the next task is the query itself, respectively:

- (1) Rows one, three, and four: using set-valued measure **Perms** applied to the set **V** of graph vertices, does the **FORALL** statement hold true, namely, that for all integers a , $0 \leq a < |V|$, we have $p(a)$ and $p(a + 1 \bmod |V|)$ are adjacent (“**ac**”)?
- (2) Rows two, three, and four: using set-valued measure **Cycless** applied to the (arbitrary) vertex 0, does the **EXISTS** statement hold true, namely that some cycle around vertex 0 has length equal to $|V|$? (A variant on this invocation would be

$$\text{MAX } (c \text{ IN Cycless}(0), \text{st}(c)) == \text{dimm}$$

that is, using the first-order continuous-valued (real-valued) quantifier **MAX** alongside the discrete-valued (integer-valued) measure **st** “size-tally”.) Note that set-valued measure **Cycless** returns a set of tuples, and then **st** of a tuple **c** is simply its length. **dimm** is $|V|$ always.

- (3) The third and fourth invocations in the screenshot equate the first two mathematical expressions. One computes them in parallel, checking they return the same number of positives:

```

/home/peter/CLionProjects/Flagcalc/ctest-build-debug/flagcalc -r 8 p=0.5 100 -a "s=EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm])))" "s=EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
RANDOMGRAPHS : r1 random graph with edgectn probability: 8, 14.000000, 100
TIMEDRUN TIMEDRUN100:
0.002374
APPLYBOOLEANCITERION APPLYBOOLEANCITERION101:
Criterion Sentence EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm]))) results of graphs:
result == 1: 57 out of 100, 0.57
result == 0: 43 out of 100, 0.43
APPLYBOOLEANCITERION APPLYBOOLEANCITERION102:
Criterion Sentence EXISTS (c IN Cycless(0), st(c) == dimm) results of graphs:
result == 1: 57 out of 100, 0.57
result == 0: 43 out of 100, 0.43
TIMEDRUN TIMEDRUN103:
0.437685
TIMEDRUN TIMEDRUNVerbosity:
0.000286
Process finished with exit code 0

```

The second uses **IFF** (“if and only if”) to check that they literally agree on each random graph (“**result == 1**” meaning result is **TRUE**):

```

/home/peter/CloneProjects/flagcalc/cmake-build-debug/flagcalc -r 8 p=0.5 100 -a "s=EXISTS (p IN Perm(V), FORALL (a IN dimm, ac[a],p[(a+1) % dimm])) IFF EXISTS (c IN Cycles(0), st(c) == dimm)" all -v i=minimal3.cfg
RANDOMGRAPHS : r1 random graph with edgcnt probability: 8, 14.000000, 100
TIMEDRUN TIMEDRUN100:
0.001895
APPLYBOOLEANCRITERION APPLYBOOLEANCRITERION101:
Criterion Sentence EXISTS (p IN Perm(V), FORALL (a IN dimm, ac[a],p[(a+1) % dimm])) IFF EXISTS (c IN Cycles(0), st(c) == dimm) results of graphs:
result == 1: 100 out of 100, 1
TIMEDRUN TIMEDRUN102:
0.00111
TIMEDRUN TimedRunVerbosity:
0.000212
Process finished with exit code 0

```

- (4) Rows five and six: using criterion (boolean-valued measure) `embedsgenerousc` it is checked whether C_8 embeds into the graph G . Please note here the first exposure to Flagcalc's language for expressing a graph: the dash at the start of the list of vertices indicates a walk, here not repeating except ending where it began (a cycle).
- (5) Rows seven and eight: using the discrete-valued (integer-valued) measure `circm` which is the graph's *circumference*, check it equals the graphs dimension $|V|$. Row eight, analogous to three and four above, checks that this characterization agrees with the earlier one.

2.2. Verifying some theorems on Hamiltonian cycles using Flagcalc. Five immediate theorems are verified by Flagcalc:

```

12 # ... Some theorems on Hamiltonian cycles:
13 # ... Dirac's Theorem (1952)
14 $PTH/flagcalc -r 8 p=0.5 100 -a s="circm == dimm IF deltam >= dimm/2" all -v i=minimal3.cfg
15
16 # ... Ore's Theorem (1960)
17 $PTH/flagcalc -r 8 p=0.5 100 -a s="circm == dimm IF FORALL (u IN V, v IN V, u < v, NOT ac(u,v) IMPLIES vdt(u) + vdt(v) >= dimm)" all -v i=minimal3.cfg
18
19 # ... Bondy-Chvátal Theorem
20 $PTH/flagcalc -r 8 p=0.5 100 -a s="circm == dimm IFF circm(Closureg) == dimm" all -v i=minimal3.cfg
21
22 # ... Tutte's Theorem (1956)
23 $PTH/flagcalc -r 8 p=0.75 100 -a isp="scripts/planarity.dat" s1="kconnc(4)" s2="planarquick" s3="circm == dimm" all -v i=minimal3.cfg
24 $PTH/flagcalc -r 12 p=0.5 10 -a isp="scripts/planarity.dat" s1="kconnc(4)" s2="planarquick" s3="circm == dimm" all -v i=minimal3.cfg
25
26 # ... Vertex cut rule
27 $PTH/flagcalc -r 8 p=0.5 100 -a s1="circm == dimm" s2="FORALL (s IN Ps(V), st(s) > 0, connc( SubgraphonUg( V SETMINUS s ) ) <= st(s))" all -v i=minimal3.cfg
28 $PTH/flagcalc -r 12 p=0.5 100 -a s1="circm == dimm" s2="FORALL (s IN Ps(V), st(s) > 0, connc( SubgraphonUg( V SETMINUS s ) ) <= st(s))" all -v i=minimal3.cfg
29

```

- (1) Dirac's Theorem. If every vertex has degree at least $|V|/2$ then the graph is Hamiltonian.
- (2) Ore's Theorem. If for any two distinct non-adjacent vertices u and v we have $\deg(u) + \deg(v) \geq |V|$ then the graph is Hamiltonian.
- (3) Bondy-Chvátal Theorem. The *closure* $cl(G)$ is Hamiltonian if and only if G is Hamiltonian. Here, $cl(G)$ is the unique graph obtained iteratively by connecting any two vertices whose degrees sum to at least $|V|$, the graph's dimension.
- (4) Tutte's Theorem. If the graph is 4-connected and planar, then it is Hamiltonian.
- (5) Vertex Cut Rule. If the graph is Hamiltonian, then for any non-empty set S of vertices, the graph induced on $V \setminus S$ has no more than $|S|$ connected components.

```

/home/peter/CLionProjects/flagcalc/cmake-build-debug/flagcalc -r 14 p=0.5 500 -a "s1=circm == dimm" "s2=FORALL (s IN Ps(V), st(s) > 0, connm( SubgraphonUg( V SETMINUS s ) ) <= st(s))" all -v i=minimal3.cfg
RANDOMGRAPHS : r1 random graph with edgecnt probability: 14, 45.500000, 500
TIMEDRUN TIMEDRUNS00:
0.006174
APPLYBOOLEANCITERION APPLYBOOLEANCITERIONS01:
Criterion Sentence circm == dimm results of graphs:
result == 1: 483 out of 500, 0.966
result == 0: 17 out of 500, 0.034
APPLYBOOLEANCITERION APPLYBOOLEANCITERIONS02:
Criterion Sentence FORALL (s IN Ps(V), st(s) > 0, connm( SubgraphonUg( V SETMINUS s ) ) <= st(s)) results of graphs:
result == 1: 483 out of 483, 1
TIMEDRUN TIMEDRUNS03:
10.5981
TIMEDRUN TiedRunVerbosity:
0.001832
Process finished with exit code 0

```

3. STATISTICS

The notion of “likely true” is no substitute for truth, but having grappled with this distinction, the purveyor of Flagcalc’s computer-assisted-proof has two observations:

- (1) One can formulate a conjecture, and then verify it on tens of millions of random graphs (or tens of thousands, or billions, with standard big-O considerations played out in the runtime and admixture with what hardware you are using; for the most part, Flagcalc is well-utilizing the CPU’s threads, but not yet the GPU). This enables “steering” the query until hard limits are found, and only then sitting down with pen and paper to prove it.
- (2) One can disprove a conjecture via finding a counter-example amongst the above-mentioned tens of millions, etc., of random graphs. Indeed, a big feature of Flagcalc is its `-g` outputting of all graphs that pass or that fail the first round(s) of querying.

Next, statistics appear throughout in many other guises, and the tool is centered on “pure math” while still entertaining interfacing with statistical tables (an analog to log tables?), or even to statistical software. More, we can look for simple correlations: what edge incidence ($p=0.5$, for example), gives us fifty-percent likelihood an n -vertex random graph is not connected (that is, has at least two disconnected parts)? What is the likelihood two randomly-chosen graphs on n vertices are isomorphic? What is the average girth of a triangle-free graph? What is the average degree $\overline{dm}$ of a planar graph on n vertices? The sky is the limit.

4. INTRICATE QUERIES

4.1. Nested quantifiers. It is not unrealistic for queries to run a dozen or more nested parentheses deep. But first a step back: queries are “staged”, meaning the workspace is whittled down one iteration at a time, `s1=...`, `s2=...`, etc.:

```
-r 25 p=0.1 1000 -a s1="FORALL (u IN V, v IN V, u < v, st(Pathss(u,v)) <= 1)" s2="conn1c" s3="treec" all -v i=minimal3.cfg
```

This first populates the workspace with one thousand random graphs on twenty-five vertices with an edge-incidence likelihood of 0.1. Then it iterates:

- (1) First, only “pass” those graphs in which for any two distinct vertices the number (“size tally `st`”) of paths between them is at most one.

- (2) Second, only “pass” those graphs in which the graph is connected.
- (3) Third, check that the combination of these two conditions means the graph is a tree (an acyclic connected graph).

4.2. Innovative keywords. Every quantifier takes two or three comma-separated inputs: a comma-separated list of variables and respective sets ranged over; an optional “criteria” to test before including the body in the results; the body itself. (There is also capacity for built-in aliasing, so as to compute a lengthy or slow computation only once, “naming” it for repeated use within the body). The first-order quantifiers are

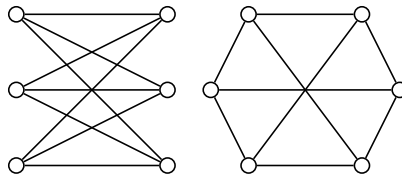
- (1) **SET**: The set of all items in the body.
- (2) **SETD**: The set of all items in the body, no time wasted checking for duplicates.
- (3) **TUPLE**: The tuple (ordered set) of all items in the body.
- (4) **FORALL**: Standard boolean-valued check that the body holds true in each case quantified over.
- (5) **EXISTS**: Standard boolean-valued check that the body holds true at least once.
- (6) **ANY**: Returns any one element from the body (like **EXISTS** except obtaining a witness).
- (7) **EXISTSUNIQUE**: Standard boolean-valued statement that the body holds true in exactly one case.
- (8) **ANYN**: Returns a set of size n consisting of arbitrarily-chosen n witnesses.
- (9) **EXISTSNUNIQUE**: Standard boolean-valued statement that there are exactly n unique body elements.
- (10) **EXISTS n** : Standard boolean-valued statement that there are at least n body elements that hold true.
- (11) **FORALL n** : Standard boolean-valued statement that there are at most n body elements that don’t hold true.
- (12) **SUM**: Continuous-valued sum of all body elements ranged over.
- (13) **MIN**: Continuous-valued least of all body elements ranged over, or *inf* if no body elements.
- (14) **MAX**: Continuous-valued greatest of all body elements ranged over, or *−inf* if no body elements.
- (15) **PRODUCT**: Continuous-valued product of all body elements ranged over, or 1 if no body elements.
- (16) **AVERAGE**: Average (mean) of all body elements, or 0 if no body elements.
- (17) **RANGE**: Maximum minus minimum of all body elements ranged over.
- (18) **TALLY**: Discrete-valued (integer-valued) sum of all values ranged over, casting each to type discrete (integer).
- (19) **COUNT**: Discrete-valued (integer-valued) count of how many values ranged over are non-zero.
- (20) **BIGCUP**: The set-theoretic union of all elements (sets) ranged over.
- (21) **BIGCUPD**: The set-theoretic union of all elements (sets) ranged over, without time-costly removal of duplicates.
- (22) **BIGCAP**: The set-theoretic intersection of all elements (sets) ranged over.

- (23) **PARTITION**: Unlike the above, takes two variables quantified over one set, and takes a relation assumed to be transitive, reflexive, and symmetric; returns a set of equivalence-classes.
- (24) **SORT**: Like the **PARTITION**, takes two variables quantified over one set, and takes a relation assumed to be transitive and anti-symmetric; returns a tuple.

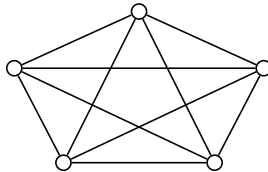
5. CASE STUDY: GRAPH MINORS AND TOPOLOGICAL MINORS

5.1. Kuratowski's Theorem. A graph is *planar* if it can be drawn in the plane without edges crossing edges. By Kuratowski's Theorem this is equivalent to the graph not having either a $K_{3,3}$ minor or a K_5 minor. This, it turns out, is equivalent to it not having either a $K_{3,3}$ topological minor or a K_5 topological minor (something not true of minors and topological minors generally).

Here is $K_{3,3}$, drawn in two fashions:



Here is K_5



Introducing, now, “stored procedures” (a name borrowed from SQL but a bit different in this context): see the excerpt from `planarity.dat`:

```

mtbool planar
~~~~~
vsetplanar ( V )
~~~~~
END

mtbool planarquick
~~~~~
NOT (hastopologicalminorc4("abc-def") OR hastopologicalminorc4("abcde"))
~~~~~
END

mtbool vsetcriticalnonplanar( mtset vs )
~~~~~
FORALL (v IN vs, vsetplanar (vs SETMINUS {v})) AND NOT vsetplanar ( vs )
~~~~~
END

mtbool criticalnonplanar
~~~~~
vsetcriticalnonplanar( V )
~~~~~
END

mtbool vsetapexgraph( mtset vs )
~~~~~
EXISTS (v IN vs, vsetplanar(vs SETMINUS {v})) AND NOT vsetplanar( vs )
~~~~~
END

```

The boolean-valued measure `vsetplanar` takes as input a set (type “`mtset`”) of vertices and returns TRUE if and only if neither $K_{3,3}$ nor K_5 embeds in it¹.

The boolean-valued measure `planar` simply calls `vsetplanar` with the entire graph as vertex set. Lastly, `planarquick` is orders of magnitude faster than `planar`, in that it uses natively-coded (C++) `hastopologicalminorc4` instead of nested stored procedures.

`criticalnonplanar` by definition is a boolean-valued measure as to whether the graph is non-planar but the removal of any one vertex makes it planar. Likewise `apexgraph` by definition is a boolean-valued measure as to whether the graph is non-planar and there is at least one vertex the removal of which makes the graph planar.

5.2. Topological minor implies minor. The code samples (doesn’t prove, but verifies in the field) a thousand random graphs on nine vertices to check that those having a topological K_5 minor also have a K_5 minor.

¹Please find `planarity.dat` in the project’s main directory under folder `scripts`, in order to inspect the Flagcalc query-native implementations of slow `vsetplanar` via stored procedure: it is an example of the versatility of the language, and indeed worth exploring which of several implementations is fastest in different scenarios.


```

182 # Diestel Prop 1.7.3
183 $PTH/flagcalc -r 9 p=0.4 1000 -a s1="hastopologicalminorc4(\"abcde\")" s2="hasminorc(\"abcde\")" all -v i=minimal3.cfg
184
185 # check for some having a topological minor of K5 but not a minor of K5
186 $PTH/flagcalc -r 8 p=0.4 100 -a s1="hasminorc(\"abcde\")" s2="NOT hastopologicalminorc4(\"abcde\")" s2="NOT hastopologicalminorc(\"abcde\")" all -v i=minimal3.cfg
187
188 # Hadwiger's conjecture (Diestel p. 184) (easy for r=3, r=4; four-color theorem for r=5, r=6; unknown for r >= 7)
189 $PTH/flagcalc -r 8 p=0.4 100 -a s1="Chit >= 3" s2="hasminorc(\"abc\")" all -v i=minimal3.cfg
190 $PTH/flagcalc -r 9 p=0.4 100 -a s1="Chit >= 4" s2="hasminorc(\"abcd\")" all -v i=minimal3.cfg
191 $PTH/flagcalc -r 10 p=0.4 100 -a s1="Chit >= 5" s2="hasminorc(\"abcde\")" all -v i=minimal3.cfg
192 # $PTH/flagcalc -r 11 p=0.4 100 -a s1="Chit >= 6" s2="hasminorc(\"abcde\")" all -v i=minimal3.cfg
193 # $PTH/flagcalc -r 12 p=0.4 100 -a s1="Chit >= 7" s2="hasminorc(\"abcdefg\")" all -v i=minimal3.cfg

```

The output shows the use of query staging, the second query correctly finding all true.

```

/home/peter/ClionProjects/flagcalc/cmake-build-debug/flagcalc -r 9 p=0.4 1000 -a s1=hastopologicalminorc4(\"abcde\") s2=hasminorc(\"abcde\") all -v i=minimal3.cfg
RANDOMGRAPHS : r1 random graph with edgecnt probability: 9, 14.40001, 1000
TIMEDRUN TIMEDRUN1000:
0.008728
APPLYBOOLEANCITERION APPLYBOOLEANCITERION1001:
Criterion Sentence hastopologicalminorc4(\"abcde\") results of graphs:
result == 1: 214 out of 1000, 0.214
result == 0: 786 out of 1000, 0.786
APPLYBOOLEANCITERION APPLYBOOLEANCITERION1002:
Criterion Sentence hasminorc(\"abcde\") results of graphs:
result == 1: 214 out of 214, 1
TIMEDRUN TIMEDRUN1003:
3.58695
TIMEDRUN TimedRunVerbosity:
0.001598
Process finished with exit code 0

```

5.3. Minor does not imply topological minor. Along the way, an additional second stage is invoked just to verify some different implementations of `hastopologicalminorc`.

```

/home/peter/ClionProjects/flagcalc/cmake-build-debug/flagcalc -r 8 p=0.4 100 -a s1=hasminorc(\"abcde\") \"s2=NOT hastopologicalminorc4(\"abcde\")\" \"s2=NOT hastopologicalminorc(\"abcde\")\" all -v i=minimal3.cfg
RANDOMGRAPHS : r1 random graph with edgecnt probability: 8, 11.20000, 100
TIMEDRUN TIMEDRUN100:
0.002037
APPLYBOOLEANCITERION APPLYBOOLEANCITERION101:
Criterion Sentence hasminorc(\"abcde\") results of graphs:
result == 0: 95 out of 100, 0.95
result == 1: 5 out of 100, 0.05
APPLYBOOLEANCITERION APPLYBOOLEANCITERION102:
Criterion Sentence NOT hastopologicalminorc4(\"abcde\") results of graphs:
result == 1: 2 out of 5, 0.4
result == 0: 3 out of 5, 0.6
APPLYBOOLEANCITERION APPLYBOOLEANCITERION103:
Criterion Sentence NOT hastopologicalminorc(\"abcde\") results of graphs:
result == 1: 2 out of 5, 0.4
result == 0: 3 out of 5, 0.6
TIMEDRUN TIMEDRUN104:
3.45495
TIMEDRUN TimedRunVerbosity:
0.000421
Process finished with exit code 0

```

Reading the output, five of one hundred random graphs on eight vertices, have a K_5 minor; of these five, two do not have a K_5 topological minor as verified using the fourth of four implementations of `hastopologicalminorc`. Finally, of these five also, two do not have a topological minor as verified using the first of four implementations of the measure `hastopologicalminorc`.

6. ISOMORPHISM CLASSES

Any query can either be preceded by or succeeded by a partitioning of the graphs on the workspace according to isomorphism class (using a “fingerprinting” feature that also sorts

them by isomorphism class). Therefore, for your slower queries, you can first partition the random graphs (or graphs drawn from a database on disk) into isomorphism classes, then run the query once per isomorphism class. For your faster queries, you can whittle down the workspace using queries, then partition the graphs that pass into isomorphism classes.

7. SUBGRAPHS

By and large the working unit is the individual graph. That said, every measure has an optional leading argument: a graph. So you can quantify over all subgraphs, for example, and compute their, say, girth using the same measure `girthm` that otherwise responds to the graph in question (i.e. without any arguments): you use `girthm(SubgraphonUg({0, 1, 2}))`.

8. CONCLUDING REMARKS

This document has been an attempt to circumnavigate the potential of FlagCalc and its querying language. Here without explanation is a screenshot of involved queries imitating theorems found in R. Diestel’s book *Graph Theory, Fifth Edition* (2017).

```

200 # Diestel 9.3.3 (Cameron et al 2016)
201
202 $PTH/flagcalc -r 5 4 10 -r 6 7.5 10 -r 7 10.5 10 -r 8 14 10 -a s="FORALL (n IN NN(9), FORALL (d IN NN(9), Chirpmet == Deltam IF (FORALL (v IN V, (vdt(v) == d) AND dimm == n AND d >= n/2 AND n >= 4 AND n % 2
203
204 $PTH/flagcalc -r 6 7.5 100 -a z="Chirpmetgreedyt - Chirpmet" all -v i=minimal3.cfg
205
206 # Diestel Conjecture p. 184 (Hadwiger 1943), Corollary 7.3.3
207
208 $PTH/flagcalc -r 8 14 10000 -a s="Chit >= 5" s2="EXISTS (so IN Ps(V), EXISTS (p IN Setpartition(so), st(p) >= 5, FORALL (q IN p, FORALL (r2 IN q, FORALL (s2 IN q, r2 != s2, EXISTS (z IN Pathss(r2,s2), z <= q)
209
210 # Diestel Corollary 7.3.2
211
212 $PTH/flagcalc -r 8 14 300 -a s="FORALL (vs IN Ps(V), NOT EXISTS (o IN Setpartition(vs), st(o) == 4, FORALL (p IN o, FORALL (q IN p, FORALL (r IN p, q != r, EXISTS (z IN Pathss(q,r), z <= p))) AND FORALL (p1
213 s2="FORALL (v1 IN V, FORALL (v2 IN V, v1 != v2 AND NOT ac(v1,v2), EXISTS (vs IN Ps(V), EXISTS (m IN Setpartition(vs), st(m) == 4, FORALL (p IN m, FORALL (q IN p, FORALL (r IN p, EXISTS (z IN Pathss(q,r), z <=
214 z3="edgcon == 2*dimm - 3" all -v i=minimal3.cfg
215
216 $PTH/flagcalc -r 8 14 300 -a s="FORALL (vs IN Ps(V), NOT EXISTS (o IN Setpartition(vs), st(o) == 4, FORALL (p IN o, FORALL (q IN p, FORALL (r IN p, q != r, EXISTS (z IN Pathss(q,r), z <= p))) AND FORALL (s1
217 s2="FORALL (no IN ne, EXISTS (vs IN Ps(V), EXISTS (m IN Setpartition(vs), st(m) == 4, FORALL (p IN m, FORALL (q IN p, FORALL (r IN p, EXISTS (z IN Pathss(q,r), z <= p))) OR (ne[0] ELT p AND ne[1] ELT p AND FE
218 z3="edgcon == 2*dimm - 3" all -v i=minimal3.cfg
219
220 $PTH/flagcalc -r 8 14 1500 -a s="FORALL (vs IN Ps(V), NOT EXISTS (o IN Setpartition(vs), st(o) == 4, FORALL (p IN o, connvsc(p,E)) AND FORALL (p1 IN o, FORALL (q1 IN o, q1 != p1, Nssc(p1,q1)))" \
221 s2="FORALL (no IN ne, EXISTS (vs IN Ps(V), EXISTS (m IN Setpartition(vs), st(m) == 4, FORALL (p IN m, connvsc(p, E CUP {ne})) AND FORALL (p2 IN m, FORALL (q2 IN m, p2 != q2, Nssc(p2,q2) OR (ne[0] ELT p2 AND n
222 z3="edgcon == 2*dimm - 3" all -v i=minimal3.cfg
223
224 # Diestel Theorem 3.3.6(1) (Global version of Menger's Theorem) Note that the native kconncc is much faster than the count of independent paths, and here that speedup is a matter of the native code being to th
225
226 $PTH/flagcalc -r 5 5 300 -a s="FORALL (k IN dimm, kconncc(k) IFF FORALL (v1 IN V, FORALL (v2 IN V, v1 != v2, EXISTS (ps IN Ps(Pathss(v1,v2))), st(ps) >= k, FORALL (p1 IN ps, FORALL (p2 IN ps, p1 != p2, FORALL (
227 $PTH/flagcalc -r 5 5 300 -a s="FORALL (k IN dimm, kconncc(k) IFF FORALL (v1 IN V, FORALL (v2 IN V, v1 != v2, EXISTS (ps IN Ps(Pathss(v1,v2))), st(ps) >= k, FORALL (p1 IN ps, FORALL (p2 IN ps, p1 != p2, p1 CAP
228 $PTH/flagcalc -r 5 5 300 -a s="FORALL (k IN dimm, FORALL (v1 IN V, FORALL (v2 IN V, v1 != v2, EXISTS (ps IN Ps(Pathss(v1,v2))), st(ps) >= k, FORALL (p1 IN ps, FORALL (p2 IN ps, p1 != p2, p1 CAP p2 <= {v1,v2}))
229
230 # Diestel Theorem 3.3.1 (Menger 1927)
231
232 $PTH/flagcalc -r 5 5 50 -a ispe="./scripts/storedprocedures.dat" s="FORALL (A IN Ps(V), FORALL (B IN Ps(V), MIN (X IN Ps(V), Separatess(A,B,X), st(X)) == DisjointApaths( A, B ))" all -v i=minimal3.cfg
233 $PTH/flagcalc -r 5 5 50 -a ispe="./scripts/storedprocedures.dat" s="FORALL (A IN Ps(V), FORALL (B IN Ps(V), MinXSeparates(A,B) == DisjointApaths( A, B ))" all -v i=minimal3.cfg

```

The most basic way to acclimate to the tool’s potential is via three means:

- (1) Read through the commented shell scripts (“testsuites”) as found on the Github page, main folder, subfolder `scripts`, and following naming convention `testsuitexxx.sh`.
- (2) Read through the PDF documents as found on the Github page, main folder, subfolder `documentation`.
- (3) Read the progress reports on the author’s LinkedIn profile feed².

²*FlagCalc* can be found at <http://github.com/corralledcode>. Note that the GUI tool “Explore” there found, includes both the command-line FlagCalc tool in Linux and Windows variants, and includes the GUI front-end for either platform, along with the extant PDF documentation. Please see the PDF in the documentation folder on how to compile FlagCalc from source for the most up-to-date version

- (4) Read through the automatic help output by running `flagcalc -h` from the command line